

Randy — Chef d'équipe, intégration, déploiement

Ta mission n'est pas de coder le plus : c'est que **l'équipe livre au jury des fonctionnalités qui marchent en ligne**. Tu décides, tu intègres, tu déploies, tu pilotes Claude et tu protèges les autres des interruptions. Judicaël te remplace quand tu dors (2 h → 5 h 30).

À lire avec : tome 2 (playbook) en entier, tome 1 chapitres 24 (déploiement) et 25 (générateur).

1. Tes responsabilités

Tu es responsable de	Concrètement
Les décisions	Triage de chaque annonce (≤ 10 min), priorités P0/P1/P2, « non » assumés
L'intégration	Relire et merger les PR, garder <code>main</code> toujours déployable
Le déploiement	<code>deploy.sh</code> toutes les 2-3 h, vérifier le site après chaque déploiement
Claude	Lancer et suivre les sessions cloud, relire leurs PR comme celles des autres
Les livrables	README, formulaire Webcup, vidéo (voix)
Le calme	Une seule voix qui tranche ; pas de débat de plus de 2 minutes

2. Les 40 premières minutes (9 h → 9 h 40)

- 9 h 00** — Copier **tout** le sujet (texte + doc de l'API de l'orga) dans Claude avec le prompt de lancement (playbook §12.1).
- 9 h 10** — Réunion de 15 min animée par Njaraniaina : tu valides le modèle de données et la répartition proposée par Claude, ou tu la corriges.
- 9 h 25** — Tu lances toi-même les `make:feature` des entités principales, `migrate`, tests, commit, push sur `main`. Tout le monde part de la même base : c'est la seule fois où l'on pousse directement sur `main`.
- 9 h 40** — Chacun a une carte « En cours ». Tu lances 1 ou 2 sessions cloud sur des fonctionnalités de base indépendantes.

3. Trier une annonce (≤ 10 min)

- `/trriage <texte de l'annonce>` dans Claude Code.
- Décider avec la grille du playbook §6 : **P0 / P1 / P2 / non**, et qui la prend.
- Le dire en une phrase sur le vocal : « A4 : export CSV, P1, Tsoa, 30 min. »
- Njaraniaina crée l'issue avec le modèle « Fonctionnalité ».

Pour décider vite, demande-toi : *le jury va-t-il voir la différence ? et est-ce que ça peut casser ce qui marche ?*

4. Merger une PR (5 min)

1. Coche verte (Pint + PHPStan + tests).
2. Lire le diff en 2 minutes :
 - routes dans `routes/features.php` (groupe `auth`) ;
 - `$this->authorize(...)` dans chaque méthode publique ;
 - pas de `user_id` / `role` / `statut` dans `#[Fillable]` ;
 - au moins un test « autre utilisateur → 403 ».
3. Si la fonctionnalité est visible : `git fetch && git checkout feat/xxx`, `php artisan migrate`, 2 minutes de test en local.
4. **Merge** (bouton *Squash and merge* ou *Merge*), puis `git checkout main && git pull`.

Conflit dans `routes/features.php`, la sidebar ou le seeder : garder **les deux** blocs. Ailleurs : l'auteur résout, Judicaël aide.

5. Déployer

```
bash ~/webcup-2026/deploy.sh
```

Le script fait : `git pull`, `composer install`, `npm ci` + `npm run build`, sauvegarde de la base, `migrate --force`, `storage:link`, `filament:assets`, `optimize`, contrôle de santé. Il s'arrête à la première erreur.

Après chaque déploiement :

1. Ouvrir le site en navigation privée, Ctrl+F5.
2. Se connecter avec le compte jury user, puis admin : rien de cassé.
3. Prévenir Njaraniaina : « Déployé : #12, #14. » Elle fait la recette.

Rythme : toutes les 2-3 h, en regroupant les merges. Jamais « on déploiera à la fin ».

6. Piloter Claude (sessions cloud)

- Environnement **Webcup**, repo `webcup-2026`. Le hook prépare tout au démarrage.
- **Une session = une fonctionnalité = une branche = une PR**. Deux sessions en parallèle au maximum, trois si le socle est stable.
- Prompt type : `playbook §12.1`. Toujours donner le **texte exact** de l'annonce et le numéro d'issue.
- Tu relis leurs PR avec la même exigence que celles de l'équipe.
- Budget : ~100 \$ de crédit cloud. Consulter l'usage toutes les 4 h ; en dessous de 30 \$, une seule session à la fois.
- Modèle : le plus puissant pour le lancement et l'audit ; Sonnet pour les fonctionnalités courantes.

7. Quand ça casse en production

```
cd ~/webcup-2026
tail -n 60 storage/logs/laravel.log
php84 artisan about
php84 artisan migrate:status
php84 artisan optimize:clear && php84 artisan optimize
```

Symptôme	Cause probable	Action
Erreur 500 partout	Cache de config, <code>.env</code>	<code>optimize:clear</code> puis <code>optimize</code>
« PHP version >= 8.x »	Bloc <code>AddHandler</code> absent de <code>public/.htaccess</code>	Le remettre (voir CLAUDE.md)
Page sans style	<code>npm run build</code> raté	Relancer <code>deploy.sh</code> , lire l'erreur
Photos invisibles	Lien <code>storage</code> absent	<code>php84 artisan storage:link</code>
Migration qui échoue	SQLite ≠ MariaDB	Corriger en local, nouvelle migration
Une fonctionnalité casse tout	—	En local : <code>git revert <sha></code> , push, <code>deploy.sh</code>

Jamais de `git reset --hard`, `migrate:fresh` ni `db:seed` sur le serveur.

8. Passation avec Judicaël (2 h et 5 h 30)

À 2 h, en 5 minutes :

- ce qui est déployé, ce qui attend en PR ;
- les annonces arrivées et les décisions prises ;
- les sessions cloud en cours et ce qu'elles font ;
- le point d'attention (bug connu, fonctionnalité risquée).

À 5 h 30, Judicaël te fait la même passation. **Le gel à 6 h est le tien.**

9. Tes checklists

Avant le gel (5 h 30)

- ☐ Toutes les PR prêtes sont mergées, les autres sont fermées ou reportées
- ☐ Déploiement fait, recette en cours
- ☐ Annonce sur le vocal : « Gel à 6 h, plus aucune nouvelle fonctionnalité »

Avant la fin (8 h 45)

- ☐ README final comité (modèle `docs/modèles/README-jury.md`)
- ☐ `docs/recap.md` à jour
- ☐ Dernier déploiement 8 h 30, site testé
- ☐ `git tag v1.0-rendu && git push --tags`
- ☐ Plus aucun push après 8 h 45

Livrables (dimanche, objectif 18 h)

- ☐ URL, comptes jury, lien GitHub, lien vidéo, récap
- ☐ Formulaire relu par Judicaël, puis envoyé
- ☐ Capture d'écran de la confirmation d'envoi